Střední průmyslová škola strojní a stavební, Tábor,

Komenského 1670

Studijní obor Technické lyceum

2D plošinová hra v Unity

Maturitní práce z odborných předmětů

Autor: **Zdeněk Lachout**

Třída: 4Lb

Školní rok: 2025/2026

Vedoucí: **MSc Ondřej Beneš**

Oponent: Jan Reindl

Hodnocení práce:

STŘEDNÍ PRŮMYSLOVÁ ŠKOLA STROJNÍ A STAVEBNÍ, TÁBOR

SPŠ TÁBOR

Zadání maturitní práce z odborných předmětů

Student:	**Zdeněk Lachout**
Studijní obor:	78-42-M/01 Technické lyceum
Zaměření:	Programování a výpočetní technika
Školní rok:	2025/2026
Třída:	4Lb
Studium:	denní

Ředitel školy Vám ve smyslu vyhlášky č. 177/2009 Sb., o bližších podmínkách ukončování vzdělávání ve středních školách maturitní zkouškou, ve znění pozdějších předpisů, stanovil tuto maturitní práci:

Téma: 2D plošinová hra v Unity

Práce bude obsahovat tyto části:

1. Různé překážky
2. Portály měnící herní mód
3. Mapu a soundtrack
4. Menu

Režim, způsob, formu a rozsah zpracování podřiďte písemnému pokynu ředitele školy č. 1/25/26 Obsah a úprava maturitní práce ze dne 1. září 2025. Tento pokyn také stanoví požadavek na počet vyhotovení maturitní práce, délku obhajoby a kritéria jejího hodnocení.

Datum zadání tématu práce: 26. září 2025

Termín odevzdání práce: 20. března 2026

Vedoucí učitel maturitní práce: MSc Ondřej Beneš

„Prohlašuji, že jsem tuto práci zpracoval samostatně a uvedl jsem veškeré použité informační prameny v souladu s citační normou ČSN ISO 690. V případě, že jsem při zpracování této práce využil nástroje umělé inteligence, výslovně to v práci uvádím. Tyto části jsou v textu jednoznačně identifikovatelné. Veškeré odborné zdroje, které byly ve výstupech AI zmíněny a bylo je možné dohledat, jsem ověřil, zařadil do seznamu použité literatury a citoval stejně, jako kdybych je nalezl sám. Prohlašuji rovněž, že tištěná a elektronická verze této práce jsou obsahově zcela shodné a že míra shody uvedená v protokolu z nástroje pro detekci plagiátů odpovídá skutečnosti.

V Táboře dne 20.3. 2026 Zdeněk Lachout

Poděkování

Rád bych poděkoval mému vedoucímu práce MSc Ondřeji Benešovi, za rady a čas, který věnoval kontrole mé práce. Dále děkuji mému oponentovi Janu Reindlovi za ochotu a oporu při vývoji mé práce.

Anotace

Tato práce se zabývá vývojem 2D hry, který obsahuje: programování, tvorbu grafického designu a vytváření hudby. Prostředí se skládá z dynamických i statických překážek a z portálu, který mění herní mód. Průchod hrou je obohacen o hudební složku, která se přizpůsobuje obtížnosti. Hlavním principem hry je ovládání objektu, který jede neustále dopředu a cílem je dostat objekt až na konec levelu. Výstupem práce je funkční 2D hra.

Anotation

This thesis deals with the development of a 2D game, which includes programming, graphic design, and music creation. The environment consists of dynamic and static obstacles and a portal that changes the game mode. The gameplay is enriched by music that adapts to the difficulty level. The main principle of the game is to control an object that moves forward continuously, with the goal of getting the object to the end of the level. The output of the thesis is a functional 2D game.

Obsah

0 Úvod 7

1 Tvorba herního prostředí 8

1.1 Tilemap 8

1.2 Grafické prvky 9

1.3 Design uživatelského rozhraní 9

1.3.1 Tvorba úvodního menu 9

1.3.2 Tvorba pause menu a počítadla pokusů 10

2 Tvorba hudby 11

3 Programování herních částí 12

3.1 Pohyb hráče – kostka 12

3.2 Pohyb hráče – raketka 14

3.3 Změna herního módu 15

3.4 Zastavení hry 18

4 Nástroje 20

4.1 Unity 20

4.2 Inkscape 20

4.3 LMMS 21

5 Závěr 22

6 Seznam použitých informačních zdrojů 23

7 Seznam obrázků 24

8 Přílohy 25

0 Úvod

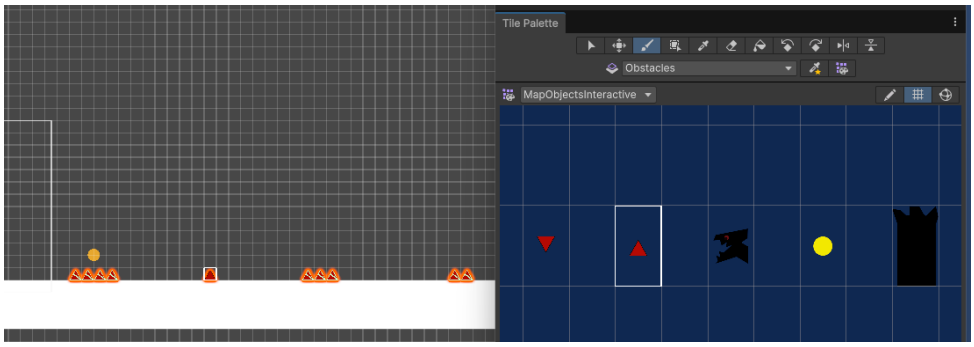
Na této 2D plošinové hře jsem se rozhodl pracovat, protože mě hry baví už od dětství a radši pracuji ve 2D prostředí. Také mě velice baví tvořit hudbu a grafiku a díky této práci jsem to všechno mohl skloubit dohromady. Největší výzvou pro mě byla tvorba přechodu mezi herními módy, zde bylo obzvlášť těžké napsat kód, co nejvíce modulárně. Pracoval jsem zde například s abstraktní třídou a abstraktními metodami. Také jsem spoustu času strávil optimalizací pohybu hráče při ovládání raketky, vždy to působilo velice nepřirozeně. Po vyzkoušení mnoha různých postupů a funkcí se mi podařilo tento problém vyřešit. Za účelem rychlé, přesné a modulární tvorby level designu jsem použil Tilemapy. Byla to pro mě úplně nová technologie a dlouho mi trvalo, než jsem se ji naučil efektivně používat. Grafický design pro mě nebyl závažný problém, vytvářel jsem pouze jednoduché geometrické tvary, které jsem následně upravoval. U tvorby hudby jsem se snažil, aby gradovala s obtížností hry a aby postavička skákala přes překážky do rytmu. Cílem mé práce bylo vytvořit funkční 2D plošinovou hru, která se podobá hře Geometry Dash.

Jádro hry jsem tvořil ve vývojářském prostředí Unity, logiku jsem programoval v jazyce C# ve VisualStudiu, hudbu jsem tvořit v nástroji LMMS a grafiku v Inkscape.

1 Tvorba herního prostředí

1.1 Tilemap

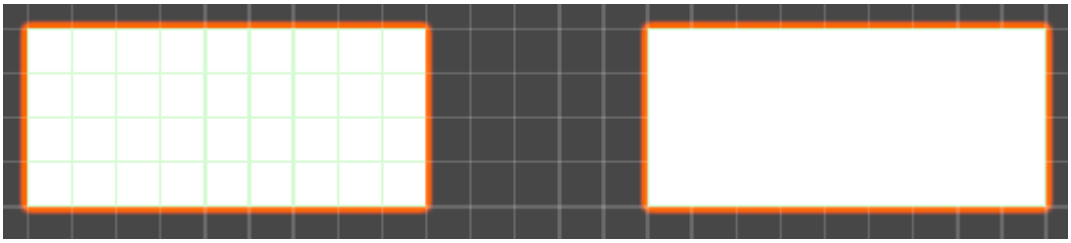
Herní prostředí jsem dělal pomocí Tilemap (komponenta, která ukládá a spravuje assety, které obsahuje (1)). Assety (předměty, které se využívají k vývoji videoher (2)) jsem vkládal s nástrojem Tile Palette. Tento nástroj vám umožňuje vkládat pomocí štětce assety přímo do hry. Velká výhoda je ta, že assety se umisťují do mřížky, takže je vaše manipulace s nimi velmi snadná a přesná.



Obrázek 1 – Práce s Tile Palette

Celkem jsem vytvořil čtyři Tilemapy, rozdělené podle jejich funkce: paletu pro zem, překážky, plošiny a pro překážky na objektech, které se hýbají. Rozdělení Tile map je nutné, protože od každé skupiny očekávám jiné chování. Toto chování primárně nastavuji pomocí Tagů a Layerů, například Tilemapa překážky má tag „překážka“, pokud tedy hráč koliduje s některým předmětem této palety, tak zemře.

Aby bylo možné kolidování musel jsem každé Tilemapě přidat komponentu Tilemap Collider 2D, která všem předmětům v rámci Tilemapy vytvoří collider. V paletě pro zem a plošiny jsem musel ještě využít komponentu Composite Collider 2D, protože zde jsem vytvářel velice velké plošiny, pomocí jednotlivých čtverců a kvůli tomu jsem pak vytvořil collider pro každý čtverec zvlášť. Composite Collider 2D umožňuje využití funkce merge, tato funkce mi celý komplikovaný collider spojila do jednoho velkého a jednoduchého.



Obrázek 2 – Porovnání funkce merge a bez funkce

1.2 Grafické prvky

Vždy jsem si nejprve vzal obyčejný tvar a poté ho upravoval. Toho jsem dosáhl tak, že jsem použil funkci objekt na cestu. Díky této funkci jsem mohl libovolně přidávat a odstraňovat uzly a měnit tak tvar. Jednotlivé uzly jsem upravoval pomocí úhlování a pomocí funkcí například: vytvořit z vybraných uzlů roh, učinit vybrané uzly hladké. Tvary jsem také několikrát kombinoval mezi sebou. Barvu jsem vybíral pomocí funkce výplň a tah. Zde je možné vybrat si odstín, sytost, jas a průhlednost barvy. Také je možné si zde vybrat z několika různých přechodů barev. Velmi zřídka jsem využil i barvu tahu, tedy vlastně barvu obrysu daného objektu. U tahu si můžete zvolit typ čáry, šířku, značky, rohy a spoje, konec čáry, pořadí.

Všechny grafické prvky jsem exportoval do hry ve formátu PNG, kvůli špatnému fungování formátu SVG v Unity, který je defaultní u Inkscape. Po exportu grafického prvku jsem musel velmi často následně upravit jeho velikost, toto jsem upravil v inspektoru daného prvku v části Sprite Mode – Pixels Per Unit. Také jsem upravoval ve Sprite Editoru pivot (bod, který je považován za střed obrazu) prvku, aby například stál na zemi a nelevitoval nad ní.

Pro objekty, které jsem chtěl animovat, jsem musel dělat více obrázků. Vždy jsem udělal první obrázek, který jsem následně okopíroval a udělal jen jemné změny, aby animace působila co nejpřirozeněji. Většinou jsem dělal pro mé jednoduché animace 2 až 3 obrázky.

Obrázek 3 – Animace překážky

1.3 Design uživatelského rozhraní

1.3.1 Tvorba úvodního menu

Začal jsem nejprve plátnem, je to nutné, všechny prvky uživatelského rozhraní zde musí být umístěny. Následně jsem přidal obrázky a tlačítka pro zapnutí a vypnutí hry. Obrázky jsem upravil rotací a velikostí. Také jsem jim musel nastavit kotvu, toto nastavení je v inspektoru a můžete si vybrat z předem připravených pozic kotvy. Nastavování kotvy je klíčové pro

správné rozmístění obrázků při změnách velikosti obrazovky, obrázek se bude vždy držet tam kde je jeho kotva, například v pravém horním rohu. Tlačítka jsem vložil do středu celého menu. Zvolil jsem font BoldPixels, který díky svému pixelovému designu působí herně. V inspektoru je u tlačítek možné nastavit funkci, která se spustí po stisknutí tlačítka. Tuto možnost jsem využil a nastavil jsem, aby jedno tlačítko hru zapnulo a druhé hru vypnulo. Barvu pozadí jsem zde vyřešil pomocí kamery, nastavil jsem, aby vykreslovala modré pozadí, pokud bych tuto kameru používal i přímo při průchodu hrou, musel bych pozadí vyřešit pomocí komponenty panel. Tento komponent se právě dá v UI využít na nastavování barvy pozadí.

Obrázek 4 – Úvodní menu

1.3.2 Tvorba pause menu a počítadla pokusů

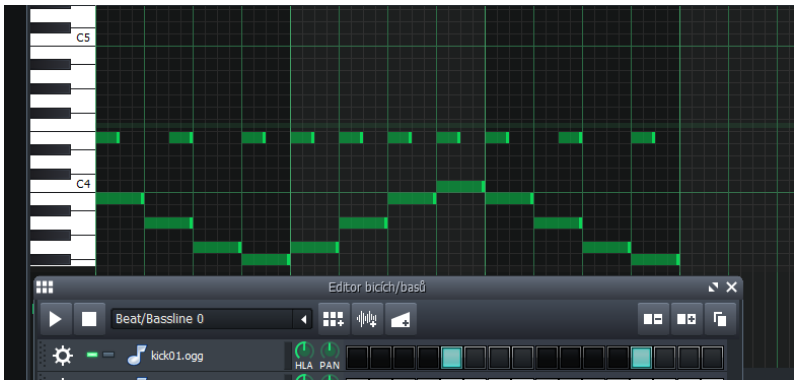
Pause menu je skoro stejné jako úvodní menu, akorát jsou zde pouze tlačítka bez obrázků a řešil jsem zde barvu pozadí pomocí komponenty panel.

Počítadlo pokusů má svoje vlastní plátno a je to obyčejný text. Zvolil jsem font LegacyRuntime, velikost fontu 33, text je v textovém bloku zarovnaný k levému hornímu rohu. Kotvu počítadla pokusů jsem nastavil též k levému hornímu rohu.

2 Tvorba hudby

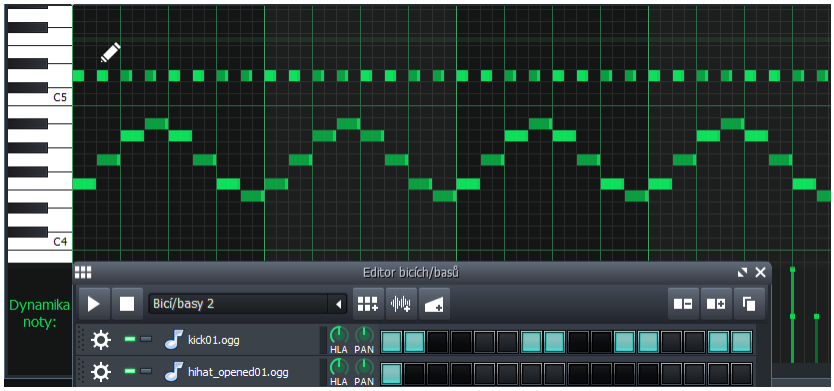
Začal jsem nastavením tempa skladby na 120 BPM (Beats Per Minute), a poté jsem začal samotnou skladbou. Můj soundtrack se skládá ze dvou prvků: melodie, kterou jsem udělal pomocí nástroje ZynAddSubFX a rytmu, který jsem vypracoval pomocí bicích. Snad nejvíce času jsem strávil zkoušením různých zvukových nástrojů v ZynAddSubFX, abych vždy našel ten, který se hodí pro moji část hry.

Ze začátku jsem zvolil velice jednoduchý rytmus, který se skládá pouze z basového bubnu. První část melodie jsem udělal s nástrojem Synth Piano2. Noty jsou zde delší než v akčnějších částech skladby, za účelem postupného budování napětí.



Obrázek 5 – První část soundtracku

U melodie byl další vývoj velice podobný, vždy jsem okopíroval melodii, která je na Obrázku 5 a dále jsem upravoval dynamiku not, výšku not, a jejich délku. Vždy jsem musel sledovat na jaké výšce skončila melodie z předešlé části a plynule navázat. Rytmus jsem upravoval přidáváním různých bicích a také jsem měnil frekvenci bubnování. U rytmu jsem se snažil o jeho podobnost v celé skladbě, pouze jsem ho v některých částech obohatil.



Obrázek 6 – Nejakčnější část soundtracku

Zde je pro porovnání obrázek, na kterém je zobrazena nejvíce akční část soundtracku.

Několikrát jsem jednotlivé části skladby poslouchal odděleně, abych se ujistil, že tato část vyhovuje mým představám. Tento úkon je možné provést kliknutím na jednu ze dvou ikonek, které jsou u všech hudebních prvků.

Obrázek 7 – Zelená a červená ikonka

Zapnutá červená ikonka izoluje vybranou část. Vypnutá zelená ikonka nám vybranou část ztiší.

3 Programování herních částí

3.1 Pohyb hráče – kostka

Hráč se pohybuje dopředu pomocí kódu rb.linearVelocityX = speed. Zkratka rb odkazuje na komponentu Rigidbody2D u hráče, tato komponenta mi dává přístup k ovládání fyziky objektu, na kterém se vyskytuje. V části linearVelocityX se dostávám do rychlosti hráče po ose x, kterou zde nastavuji pomocí proměnné speed.

Skok a rotaci hráče mám v podmínce:

```
if ((Input.GetKeyDown(KeyCode.Space) || Input.GetMouseButtonDown(0)) && isGrounded && PauseMenuCaller.gamePaused == false)
{
    Jump();
    StartRotation();

}
```

Obrázek 8 – Podmínka rotace a skok

Podmínka říká, pokud jsem stiskl tlačítko mezerník, nebo jsem stiskl levé tlačítko na myši a zároveň je bool isGrounded pravda a zároveň bool gamePaused ze skriptu PauseMenuCaller je lež, tak spusť metodu Jump a metodu StartRotation.

```
void Jump()
{
    rb.linearVelocityY = jumpForce;
    isGrounded = false;
}
```

Obrázek 9 – Metoda Jump

V metodě Jump nastavuji rychlost tentokrát na ose y na proměnnou jumpForce a bool isGrounded na false, protože kdybych nechal isGrounded na true, hra by si myslela, že je hráč na zemi a mohl bych tím pádem znovu skočit ve vzduchu.

```
void StartRotation()
{
    targetAngle -= 180f;
    isRotating = true;
}
```

```
if (isRotating)
{
    playerTransform.Rotate(0f, 0f, -rotationSpeed * Time.deltaTime);

    if (Mathf.Abs(Mathf.DeltaAngle(playerTransform.eulerAngles.z, targetAngle)) < 5f)
    {
        playerTransform.rotation = Quaternion.Euler(0, 0, targetAngle);

        isRotating = false;

    }
}
```

Obrázek 10 – Metoda StartRotation

Zde nastavuji proměnnou targetAngle na -= 180f (f je označení pro float je to druh proměnné, která umí pracovat s desetinnými čísly). Zvolil jsem záporné číslo kvůli směru rotace doprava. Na začátku podmínky if (isRotating) odkazuji na transform hráče, což je komponenta, díky které mohu přistupovat k pozici, rotaci a scale hráče. Používám funkci Rotate, která příjímá argumenty, okolo jakých os v pořadí (x,y,z) má rotovat objekt. Já jsem zde zvolil rotaci okolo osy Z o zápornou hodnotu v proměnné rotationSpeed, následně tuto proměnnou násobím pomocí Time.deltaTime. Důvodem tohoto násobení je optimalizace, celý tento kód, který zde popisuji se totiž provádí každý snímek, tudíž u výkonnějších počítačů by se kód provedl mnohem častěji kvůli většímu počtu snímků, to by mohlo způsobit například různě rychlou rotaci na různých počítačích. Time.deltaTime je vlastně čas od posledního snímku do současného snímku, tudíž řeší naši problematiku, protože pokud máte 300 snímků za vteřinu tak čas mezi dvěma snímky je přibližně 0.0033 sekundy, takže se 300krát vynásobí hodnota – rotationSpeed hodnotou 0.0033, pokud bychom měli pouze 60 snímků za vteřinu tak by se nám hodnota – rotationSpeed 60krát vynásobila hodnotou přibližně 0.016 (3).

V další podmínce používám Mathf, což je struktura, která obsahuje několik základních matematických funkcí. Konkrétně používám funkci Abs, která udělá z hodnoty v závorkách absolutní hodnotu, také používám funkci DeltaAngle, která vypočítá nejkratší vzdálenost mezi dvěma úhly, zde se jedná o úhel rotace hráče na ose z a o cílový úhel (4). Na konci

podmínky kontroluji, zda je rozdíl v absolutní hodnotě mezi dvěma úhly menší než 5. Uvnitř podmínky nastavuji rotaci hráče na cílový úhel a bool isRotating na false.

3.2 Pohyb hráče – raketka

```
if ((Input.GetKey(KeyCode.Space) || Input.GetMouseButton(0)) && PauseMenuCaller.gamePaused == false)
{
    thrustVector = new Vector2(0, Mathf.MoveTowards(rb.linearVelocityY, Maxthrust ,thrust * Time.deltaTime));
    rb.linearVelocityY = thrustVector.y;
    rotationUp = true;

    targetAngle = 30f;
```

Obrázek 11 – Podmínka pro pohyb raketky

V této podmínce kontroluji, jestli hráč drží mezerník nebo levé tlačítko myši a zároveň jestli není hra zrovna stopnutá. Následně nastavuji proměnou thrustVector, která je typu vector a nastavuji jí x komponentu na 0 a y nastavuji pomocí funkce Mathf.MoveTowards, která přijímá 3 argumenty: aktuální hodnotu – v mém případě rychlost raketky, maximální hodnotu – maximální rychlost raketky a hodnotu o kterou se bude aktuální hodnota přibližovat k té cílové – proměnná thrust vynásobená Time.deltaTimem(5). Poté nastavuji hráčovu rychlost na y na velikost y složkou proměnné thrustVector, dále nastavuji proměnnou rotationUp na true, protože když hráč letí nahoru tak chci, aby jeho rotace směřovala nahoru, poté nastavuji cílový úhel rotace na 30.

```
else
{
    thrustVector = new Vector2(0, Mathf.MoveTowards(rb.linearVelocityY, -Maxthrust, thrust * Time.deltaTime));
    rb.linearVelocityY = thrustVector.y;

    rotationDown = true;

    targetAngle = -30f;
```

Obrázek 12 – Pohyb raketka, pokud padá

Pokud se předešlá podmínka nenaplní tak nastavuji rychlost raketky do záporných hodnot a raketka díky tomu klesá. Nastavuji rotationDown na true, protože chci přepnout z rotace směrem nahoru do rotace směrem dolu a také nastavuji cílový úhel rotace na -30, kvůli směru dolů.

```
if (time >= 0f && rotationDown)
{
    time = 0f;
}

if (time <= 1)
{
    time = Time.deltaTime;
}
```

```
if (time >= 0f && rotationUp)
{
    time = 0f;
}

if (time <= 1)
{
    time = Time.deltaTime;
}
```

Obrázek 13 – Počítání času během rotace raketky

Pro regulaci rychlosti rotace používám proměnou čas. V kódu nalevo se čas počítá, pokud hráč letí nahoru, v kódu napravo se čas počítá, pokud hráč padá dolů. Pokud hráč přešel z jedné rotace na druhou, tak se čas počítá zase od 0, tento případ je podchycen v první podmínce. Ve druhé podmínce pouze reguluji hodnotu času, aby nikdy nebyla větší než jedna.

```
Quaternion targetRotation = Quaternion.Euler(0f, 0f, targetAngle);

playerTransform.rotation = Quaternion.Lerp(playerTransform.rotation, targetRotation,  time * 2.5f);
```

Obrázek 14 – Nastavování rotace raketky

Zde nastavuji hodnotu cílové rotace targetRotation, která je typu Quaternion. Quaternion v Unity představuje rotace. Pomocí funkce Quaternion.Euler mohu jednoduše nastavit okolo, které osy chci rotovat o jakou hodnotu. Rotaci raketky nastavuji pomocí funkce Qauternion.Lerp, která vezme aktuální rotaci raketky a pomocí proměnné time ji celou dobu rovnoměrně přibližuje k hodnotě proměnné targetRotation.

3.3 Změna herního módu

```
using UnityEngine;

Skript Unity | Počet odkazů: 6
public abstract class PlayerForm : MonoBehaviour
{

    protected Rigidbody2D rb;
    protected PolygonCollider2D polygonCollider2D;
    protected Transform playerTransform;

   // public virtual void FixedUpdateForm() { }

    Počet odkazů: 2
    public virtual void Init(Rigidbody2D rigidbody, PolygonCollider2D polygoncollider2D, Transform playertransform)
    {
        rb = rigidbody;
        polygonCollider2D = polygoncollider2D;
        playerTransform = playertransform;
    }

    Počet odkazů: 3
    public abstract void UpdateForm();     // Abstraktní metoda, nemá tělo
    Počet odkazů: 3
    public abstract void OnEnter();
    Počet odkazů: 3
    public abstract void OnExit();

    Zpráva Unity | Počet odkazů: 3
    public virtual void OnCollisionEnter2D(Collision2D collision)
    { }

    Zpráva Unity | Počet odkazů: 2
    public virtual void OnCollisionExit2D(Collision2D collision)
    { }

}
```

Obrázek 15 – Kostra hráče

Nejprve jsem zde definoval komponenty, které bude jakákoliv forma hráče potřebovat a dal jsem před ně slovo protected, to umožnuje využívání těchto komponent skriptům, které dědí

z této třídy (6). Následně vytvářím metodu Init, díky které nastavím komponenty na ty, které má ve svých argumentech. Následně jsem vytvořil tři abstraktní metody, které si každá forma nadefinuje sama a také jsem zde vytvořil dvě metody na detekci kolizí.

```
public class PlayerFormManager : MonoBehaviour
{
    private Rigidbody2D rb;
    private PolygonCollider2D polygonCollider2D;
    private Transform playerTransform;

    public PlayerForm cubeForm;
    public PlayerForm rocketForm;
    public PlayerForm currentForm;
```

Obrázek 16 – Definice potřebných komponent

V dalším skriptu PlayerFormManager, který slouží hlavně jako přepínač hráčových forem, jsem definoval hlavní komponenty.

```
Zpráva Unity | Počet odkazů: 0
void Start()
{
    rb = GetComponent<Rigidbody2D>();
    polygonCollider2D = GetComponent<PolygonCollider2D>();
    playerTransform = GetComponent<Transform>();

    cubeForm.Init(rb, polygonCollider2D, playerTransform);
    rocketForm.Init(rb, polygonCollider2D, playerTransform);


    SetForm(cubeForm);
}
```

Obrázek 17 – Připsání argumentů k formám

Zde v metodě Start, která se spouští jen jednou na začátku programu připisuji hráčovi komponenty z inspektoru pomocí metody GetComponent. Do metody Init jsem následně přepsal hráčovi komponenty. Také pomocí metody SetForm nastavuji hráčovu formu na defaultní tedy na kostku.

```
Zpráva Unity | Počet odkazů: 0
void Update()
{

    if (currentForm != null)
    {
        currentForm.UpdateForm();
    }
}

Počet odkazů: 3
public void SetForm(PlayerForm newForm)
{
    if (currentForm != null)
    {
        currentForm.OnExit();
    }

    currentForm = newForm;
    currentForm.OnEnter();
}

Zpráva Unity | Počet odkazů: 0
public void OnTriggerEnter2D(Collider2D collision)
{

    if (collision.CompareTag("RocketPortal"))
    {
        SetForm(rocketForm);
    }
    else if (collision.CompareTag("CubePortal"))
    {
        SetForm(cubeForm);

    }
}
```

Obrázek 18 – Přepínání forem

V metodě Update, která se volá každý snímek kontroluji, jestli není aktuální forma prázdná, pokud ne volá se metoda UpdateForm, která ve skriptech pro ovládání jednotlivých forem zastupuje metodu Update. V metodě SetForm, přijímám argument, který určí novou formu a uvnitř metody spouštím další metodu OnExit tedy co se má stát, když se forma mění na jinou a následně tedy nastavuji aktuální formu na novou a spouštím metodu OnEnter, která určuje, co se má dít při zapnutí nové formy.

V metodě OnTriggerEnter, kontroluji, jestli se hráč nestřetl s nějakým objektem. Aby tato metoda fungovala musí mít hráč v inspektoru komponentu Collider 2D já používám Polygon Collider 2D, který se automaticky přizpůsobí tvaru vašeho objektu. Ten Collider musí mít v inspektoru zaškrtnuté políčko Is Trigger, díky tomu ten Collider kolize pouze detekuje a fyzicky nekoliduje, což se mi hodí, protože v podmínkách kontroluji, jestli se hráč nestřetl s objektem, který má tag RocketPortal anebo CubePortal tedy zda se nestřetl s portálem a já potřebuji, aby hráč mohl portálem projít, a ne aby s ním kolidoval. Pokud jedna z podmínek je pravda, tak se nastaví nová forma.

```
Zpráva Unity | Počet odkazů: 0
private void OnCollisionEnter2D(Collision2D collision)
{
    if (currentForm != null)
    {
        currentForm.OnCollisionEnter2D(collision);
    }
}

Zpráva Unity | Počet odkazů: 0
private void OnCollisionExit2D(Collision2D collision)
{
    if (currentForm != null)
    {
        currentForm.OnCollisionExit2D(collision);
    }
}
```

Obrázek 19 – Informace o kolizi

V těchto funkcích se kontroluje, jestli hráč s něčím nekolidoval. Aktuální formě předávám informace o kolizích hráče, díky metodě OnCollisonEnter2D a OnCollisionExit2D, které jsem vytvořil ve skriptu PlayerForm.

3.4 Zastavení hry

```
using UnityEngine;

Skript Unity (počet odkazů na prostředky: 1) | Počet odkazů: 3
public class PauseMenuCaller : MonoBehaviour
{
    public Canvas pauseMenu;
    public PlayerMovement playerMovement;
    public AudioSource soundTrack;
    // Update is called once per frame

    public static bool gamePaused = false;

    Zpráva Unity | Počet odkazů: 0
    void Update()
    {
        if (Input.GetKeyDown(KeyCode.Escape))
        {
            Cursor.visible = true;
            Cursor.lockState = CursorLockMode.None;
            soundTrack.Pause();

            pauseMenu.enabled = true;
            gamePaused = true;
            Time.timeScale = 0;

        }
    }
}
```

Obrázek 20 – Pause Menu

Tento kód mi zajišťuje zastavení hry při stisku klávesy Escape. Na horních řádcích si definuji public proměnné, které slouží jako odkazy na už existující komponenty ve scéně. Také zde definuji proměnnou gamePaused, která je bool a je statická. Statická proměnná je proměnná, která nepatří konkrétnímu objektu ale celé třídě (v mém případě třídě PauseMenuCaller) to znamená, že napříč všemi objekty dané třídy je tato proměnná pouze jedna a má pro všechny objekty stejnou hodnotu. Díky tomu a také díky tomu, že je proměnná public na ní mohu

z ostatních skriptů odkazovat rovnou, stačí napsat PauseMenuCaller.gamePaused a nemusím nejprve vytvářet odkaz na skript.

V metodě Update kontroluji, jestli hráč nestiskl tlačítko Escape. Při stisknutí Escapu se na obrazovce ukáže Menu, zviditelní a povolí se kurzor, stopne se soundtrack a ve hře se nastaví čas na 0, jinak by sice hráč viděl menu, ale na pozadí by hra stále běžela.

```
public class MainMenuScript : MonoBehaviour
{
    public Canvas pauseMenu;
    public AudioSource soundTrack;

    Počet odkazů: 0
    public void PlayGame()
    {
        SceneManager.LoadScene(1);

    }

    Počet odkazů: 0
    public void QuitGame()
    {
        Application.Quit();
    }

    Počet odkazů: 0
    public void ResumeGame()
    {
        pauseMenu.enabled = false;
        Time.timeScale = 1;
        PauseMenuCaller.gamePaused = false;
        soundTrack.Play();
        StartCoroutine(HideCursor());

    }

    Počet odkazů: 1
    IEnumerator HideCursor()
    {

        yield return new WaitForSeconds(1f);

        Cursor.visible = false;
        Cursor.lockState = CursorLockMode.Locked;

    }
}
```

Obrázek 21 – MainMenu

Tento skript nastavuje samotnou funkci menu, vlastně co jaké tlačítko dělá. Ve hře mám dvě menu hlavní – toto se zobrazuje při spuštění hry a vedlejší menu – toto zobrazuje při stopnutí hry. Skript na obrázku 21 ovládá obě menu, protože si jsou velice podobná. Metoda playGame mi načte scénu, která má build index 1 (v mém případě je to scéna, ve které je gameplay). V metodě QuitGame používám třídu Application díky které mohu vypnout hru (pouze v buildu). V metodě ResumeGame nastavuji, věci potřebné k tomu, aby mohla hra opět běžet bez problémů (například, nastavuji, aby běžel čas a hrála opět hudba…). Komplikovanější bylo vyřešit problém s kurzorem. Pro zmizení kurzoru jsem musel použít

korutinu, protože bez ní mi při stisknutí tlačítka pro pokračování hry kurzor ihned zmizel a nic se nestalo. Korutina je metoda, která se může v průběhu zastavit a po uplynutí určitého času opět pokračovat. Korutina se zastaví, jakmile kód narazí na řádek s příkazem yield return, zde jsem definoval čas, po který nebude kód pokračovat.

4 Nástroje

4.1 Unity

Unity je herní engine, vyvinutý společností Unity Technologies. V Unity lze vytvářet 2D i 3D hry. Podporuje programování v jazyce C#. Unity je multiplatformní software, tedy podporuje různé operační systémy např. Microsoft Windows, macOS a Linux. Mezi hlavní komponenty patří:

Vykreslovací engine – Tato komponenta zajišťuje vykreslování různých tvarů, materiálů, světel atd.

Fyzikální engine – Simuluje fyziku např. gravitace, pohybu atd. Aby byl objekt ve hře řízený fyzikálním enginem musí obsahovat komponentu Rigidbody (při vývoji 2D hry Rigidbody2D)

Skriptování – Díky skriptům je možné celou hru oživit, pomocí skriptů lze např. kontrolovat kolize, ovlivňovat fyziku hry, ukládat data, číst vstupy od hráče. Programovacím jazykem je v Unity C# (7).

4.2 Inkscape

Inkscape je editor vektorové grafiky vytvořený společností The Inkscape Team. Inkscape je možné použít k tvorbě základních vektorových tvarů (např. čtverce, obdélníky, elipsy).

Základní nástroje:

Nástroj pro výběr – Umožňuje přesun objektu a změnu jeho velikosti. Také s ním přesouváme tvary na našem plátně na různé úrovně.

Nástroj pro editaci uzlů – Díky němu je možné tvar různě ohýbat. Můžeme také vybrané uzly udělat hladké, nebo z nich udělat např. roh.

Nástroj Pero – Pomáhá vytvořit tvary pomocí Bézierových křivek a přímek.

Nástroj pro tvorbu čtverců – Slouží k tvorbě různě velikých čtverců či obdélníků. Je u nich možné nastavení rozměrů v pixelech, milimetrech, centimetrech atd (8).

4.3 LMMS

LMMS je program pro tvorbu hudby. Poprvé byl vydán v roce 2004. Vytváření hudby umožňuje aranžováním samplů, syntézou zvuků, zadáváním not, hraním na MIDI klávesnici. Ukládání projektů lze v komprimovaném formátu MMPZ nebo nekomprimovaném formátu MMP (9).

5 Závěr

Výstupem mé maturitní práce je funkční 2D plošinová hra, která splňuje všechny body ze zadání a je hratelností podobná hře Geometry Dash. Tvorba hry mě zdokonalila v objektově orientovaném programování v jazyce C#, díky čemuž jsem byl schopen vyřešit problém s modulárním kódem pro přepínání jednotlivých herních módů. Veliké zlepšení jsem také udělal u level designu využitím Tilemap díky kterým jsem byl schopen vytvořit precizní level design, který je u těchto her velmi důležitý. Propojení hudby a level designu mi umožnilo vytvořit rytmický průchod hrou, což je pro mou hru esenciální. Moji hru lze využít především jako volnočasovou aktivitu. Do budoucna by hra šla jednoduše rozšířit o další herní módy a o nové levely s novou hudbou.

6 Seznam použitých informačních zdrojů

1. *Tilemap*. Online. Unity Documentation. 2019. Dostupné z: https://docs.unity3d.com/2019.1/Documentation/Manual/class-Tilemap.html. [cit. 2026-03-11].

2. *What Are Assets in Game Design?* Online. RetroStyle Games. 2026. Dostupné z: https://retrostylegames.com/blog/what-are-assets-in-game-design/. [cit. 2026-03-11].

3. *Time.deltaTime*. Online. Unity Documentation. 2026. Dostupné z: https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Time-deltaTime.html [cit. 2026-03-11].

4. *Mathf.DeltaAngle*. Online. Unity Documentation. 2026. Dostupné z: https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Mathf.DeltaAngle.html. [cit. 2026-03-11].

5. *Mathf.MoveTowards*. Online. Unity Documentation. 2026. Dostupné z: https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Mathf.MoveTowards.html. [cit. 2026-03-11].

6. *What is the difference between private and protected?* Online. Unity Discussions. 2026. Dostupné z: https://discussions.unity.com/t/what-is-the-difference-between-private-and-protected/91775. [cit. 2026-03-11].

7. *Unity (herní engine)*. Online. In: Wikipedia: the free encyclopedia. San Francisco (CA): Wikimedia Foundation, 2025. Dostupné z: https://cs.wikipedia.org/wiki/Unity_(hern%C3%AD_engine). [cit. 2026-03-11].

8. *Inkscape*. Online. In: Wikipedia: the free encyclopedia. San Francisco (CA): Wikimedia Foundation, 2026. Dostupné z: https://cs.wikipedia.org/wiki/Inkscape. [cit. 2026-03-11].

9. *LMMS*. Online. In: Wikipedia: the free encyclopedia. San Francisco (CA): Wikimedia Foundation, 2024. Dostupné z: https://cs.wikipedia.org/wiki/LMMS. [cit. 2026-03-11].

7 Seznam obrázků

Obrázek 1 – Práce s Tile Palette 8

Obrázek 2 – Porovnání funkce merge a bez funkce 8

Obrázek 3 – Animace překážky 9

Obrázek 4 – Úvodní menu 10

Obrázek 5 – První část soundtracku 11

Obrázek 6 – Nejakčnější část soundtracku 11

Obrázek 7 – Zelená a červená ikonka 12

Obrázek 8 – Podmínka rotace a skok 12

Obrázek 9 – Metoda Jump 12

Obrázek 10 – Metoda StartRotation 13

Obrázek 11 – Podmínka pro pohyb raketky 14

Obrázek 12 – Pohyb raketka, pokud padá 14

Obrázek 13 – Počítání času během rotace raketky 14

Obrázek 14 – Nastavování rotace raketky 15

Obrázek 15 – Kostra hráče 15

Obrázek 16 – Definice potřebných komponent 16

Obrázek 17 – Připsání argumentů k formám 16

Obrázek 18 – Přepínání forem 17

Obrázek 19 – Informace o kolizi 18

Obrázek 20 – Pause Menu 18

Obrázek 21 – MainMenu 19

8 Přílohy

1. Projekt v enginu Unity (přiloženo pouze elektronicky na TEAMS – OneDrive)
2. Spustitelná verze hry (přiloženo pouze elektronicky na TEAMS – OneDrive)
3. Zpráva z Odevzdej.cz

Příloha 3

OD

odevzdej@fi.muni.cz odevzdej@fi.muni.cz

14. 3. 2026, 11:51

Komu: z.lachy1@seznam.cz

Odevzdej.cz: Podobnosti Vašich souborů byly spočítány

Shrnout konverzaci | Shrnout e-mail

Odevzdej.cz: Podobnosti Vašich souborů byly spočítány

Zpracovávaný dokument:
==========
E-mail: z.lachy1@seznam.cz
Jméno souboru: Dokument_MP_-_Lachout__odevzdej.cz_1_.docx
Vloženo: 13. 3. 2026
Vloženo z IP: 109.164.52.238
Podobnosti:
https://odevzdej.cz/...ocx
Výsledek porovnávání dokumentu:
==========
K vloženému souboru nebyl v databázi nalezen žádný podobný dokument.

Nález podobností byl vyhodnocen algoritmem zaměřeným na odhalení i více přeformulovaných textů, nasazeným v 5/2022.

Pro přístup k nalezeným podobnostem je potřebné být přihlášen do systému Odevzdej.cz (http://odevzdej.cz/).

Obvyklé dotazy uživatelů jsou zpracovány v FAQ (často kladené otázky) http://odevzdej.cz/...faq.
Zpracováno automaticky Informačním systémem Odevzdej.cz.